PROJECT-2: TUGAS INDIVIDUAL EKSPLORASI DBMS NOSQL GRAPH DATABASE NEO4J

IF4040 Pemodelan Data Lanjut



Disusun oleh:

Anella Utari Gunadi

13523078

SEKOLAH TINGGI ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

BAB I
DESKRIPSI UMUM

1.1. Sejarah

Neo4j adalah *native graph database* yang dirancang untuk menyimpan, mengelola, dan menganalisis data berdasarkan hubungan antar-entitas. Neo4j menggunakan model *property graph* yang terdiri dari *node, relationship,* dan *property*. Model ini membuat hubungan antar data menjadi bagian utama dari struktur basis data sehingga proses pencarian dapat dilakukan lebih cepat.

Pada tahun 2007, Neo4j resmi dibentuk sebagai perusahaan di Swedia. Neo4j menjadi salah satu *graph database* pertama yang menyediakan kemampuan *native graph storage* dan *native graph processing*. Di tahun yang sama, Neo4j dirilis sebagai perangkat lunak *open source* sehingga komunitas pengembang dapat menggunakannya secara luas. Tahun 2011, Neo4j memperkenalkan bahasa *query* khusus yang bernama Cypher dan pada tahun 2013, Neo4j memperkenalkan konsep *labeled property graph* yang memungkinkan pengguna memberikan label pada node untuk mempermudah pengelolaan data.

1.2. Penggunaan Utama

Salah satu penggunaan utama Neo4j adalah untuk membangun sistem rekomendasi berbasis hubungan antar data. Neo4j dapat memodelkan hubungan antar pengguna, produk, konten, lokasi, maupun aktivitas pengguna. Berdasarkan pola hubungan tersebut, sistem dapat dengan mudah menemukan suatu produk yang disukai pengguna berdasarkan perilaku dan karakteristik suatu pengguna.

Neo4j juga dapat digunakan untuk mendeteksi penipuan (*fraud detection)*, misalnya dalam sektor keuangan untuk mendeteksi aktivitas penipuan karena kasus *fraud* biasanya melibatkan jaringan hubungan yang kompleks. Dengan menggunakan analisis *graph*, sistem dapat menemukan pola yang mencurigakan, misalnya terdapat beberapa akun berbeda yang menggunakan perangkat atau rekening yang sama.

1.3. *Success Story*

Salah satu *success story* dari penggunaan *graph database* adalah perusahaan eBay. Perusahaan ini merupakan salah satu perusahaan *e-commerce* terbesar di dunia. Tantangan utama eBay adalah bagaimana memahami hubungan antara pengguna, produk, pencarian, transaksi, dan preferensi pengguna secara cepat. Sebelumnya, sistem berbasis RDBMS mengalami kesulitan ketika harus melakukan analisis hubungan yang kompleks, misalnya ingin melihat pola perilaku pelanggan.

Dengan menggunakan Neo4j, eBay membangun sistem berbasis graph untuk memahami hubungan antar data tersebut. Model *graph* memungkinkan eBay melakukan pencarian pola hubungan secara lebih efisien sehingga dapat meningkatkan kemampuan personalisasi dan rekomendasi produk kepada pengguna.

BAB II
ARSITEKTUR SISTEM

2.1. Model Data Neo4j

Pada *graph database*, terdapat 3 *property graph model*, yaitu *node, relationship*, dan *property*. Di *graph model*, representasi objek disebut sebagai *node*. *Node* adalah objek atau entitas yang ingin disimpan di *database*. Dalam RDBMS, *node* ini biasanya dikenal dengan baris dalam tabel.

Relationship adalah hubungan antara dua *node*. *Relationship* ini sama seperti relasi yang terdapat pada ERD. Namun pada RDBMS, relasi antar tabel didefinisikan sebagai *foreign key*. Sedangkan pada *graph database*, relasi digambarkan dengan *relationship* untuk menghubungkan antar *node*/data. *Relationship* juga dapat menyimpan informasi tambahan.

Property adalah informasi atau atribut yang menjelaskan *node* atau *relationship*. Pada model relasional, *property* ini biasa disebut dengan atribut pada suatu tabel. *Property* ini dapat dipakai sebagai informasi/atribut pada *node* dan juga pada *relationship*.

2.2. Perbandingan *Graph Model* dengan Relasional

Berikut adalah beberapa perbedaan antara model relasional dengan model *graph* Neo4j:

Model Relasional	**Model *Graph* (Neo4j)**
Tabel	*Node*
Primary Key	*Identifier*
Foreign Key	*Relationship*
Atribut	*Property*
JOIN	*Graph Traversal*
Query SQL	*Query* Cypher

2.3. Arsitektur Sistem Neo4j

Arsitektur sistem Neo4j memiliki beberapa lapisan, lapisan dari yang terluar hingga terdalam adalah *application layer, query interface,* Neo4j *database engine, storage engine,* dan *graph engine. Application layer* adalah bagian paling atas yang digunakan pengguna atau aplikasi untuk berinteraksi dengan Neo4j. Aplikasi tidak langsung mengakses penyimpanan data, tetapi mengirim permintaan melalui *query* ke Neo4j.

Lapisan selanjutnya adalah *query interface* atau *Cypher Query Language*. Cypher adalah bahasa *query* khusus Neo4j yang digunakan untuk mengambil, membuat, mengubah dan menghapus data *graph*. Konsepnya hampir sama seperti SQL pada *database* relasional dengan sintaks yang dibuat untuk membaca hubungan antar data.

Lapisan selanjutnya adalah Neo4j *database engine* yang menjadi bagian utama untuk menjalankan proses *database*. Terdapat beberapa komponen, yaitu *query processor, transaction manager,* dan *security manager. Query processor* berfungsi untuk membaca *query* Cypher, mengubah *query* menjadi rencana eksekusi, dan mencari data yang sesuai. *Transaction manager* berfungsi untuk menjaga agar perubahan data tetap konsisten dengan mengikuti prinsip ACID. *Security manager* berfungsi untuk mengatur keamanan akses, misalnya siapa yang dapat membaca dan mengubah data.

Lapisan selanjutnya adalah *storage engine* yang bertugas untuk menyimpan struktur *graph* secara fisik di media penyimpanan. Neo4j menyimpan komponen *graph* secara terpisah. Komponen-komponen tersebut adalah *node store* yang menyimpan informasi *node*, *relationship store* yang menyimpan hubungan antar *node*, *property store* yang menyimpan nilai atribut, dan *index store* untuk mempercepat pencarian data.

Lapisan paling terakhir adalah *graph storage* yang berfungsi sebagai tempat data *graph* disimpan. Pada lapisan ini, data tidak disimpan sebagai tabel, tetapi disimpan sebagai *node, relationship*, dan *property*. Struktur ini yang membedakan *graph database* dengan *database* lainnya dan dapat melakukan pencarian hubungan dengan cepat.

Alur kerja Neo4j secara garis besar adalah pertama pengguna mengirim Cypher *query* pada Neo4j. *Application layer* mengirim *query* tersebut untuk diproses oleh lapisan selanjutnya. Kemudian, *query processor* akan membaca pola hubungan sesuai dengan *query* tersebut dan membuat *execution plan* sebelum pengambilan data. Setelah itu, *storage engine* akan mengambil data sesuai dengan *execution plan* yang telah dibuat oleh *query processor*. Hasil data yang diambil oleh *storage engine* kemudian dikembalikan ke aplikasi dan dapat dilihat oleh pengguna.

BAB III
METODOLOGI PEMODELAN DATA

3.1. Identifikasi *Node*

Tahap awal dalam proses pemodelan *graph database* adalah melakukan identifikasi terhadap objek utama yang direpresentasikan sebagai *node*. *Node* ini menggambarkan suatu entitas yang memiliki karakteristik tertentu dan dapat menyimpan informasi dalam bentuk *property*. Proses ini dilakukan dengan menentukan objek yang memiliki peran penting dalam sistem serta memiliki data yang perlu disimpan dan dikelola.

Dalam proses ini, setiap entitas harus dipilih berdasarkan kebutuhan sistem agar struktur *graph* yang dibuat tidak mengandung data yang tidak diperlukan. Berbeda dengan model relasional yang menggunakan tabel, Neo4j menggunakan *node* sebagai unit utama penyimpanan data. *Node* yang telah ditentukan selanjutnya akan menjadi dasar dalam membangun hubungan antar data pada tahap berikutnya.

3.2. Identifikasi *Relationship*

Setelah menentukan *node*, tahap selanjutnya adalah melakukan identifikasi *relationship*. *Relationship* digunakan untuk menunjukkan bagaimana suatu entitas memiliki keterkaitan dengan entitas lainnya dalam suatu domain. Pada Neo4j, *relationship* merupakan komponen utama dalam struktur *graph* karena hubungan antar data disimpan secara langsung, bukan melalui mekanisme *foreign key* seperti pada RDBMS.

Relationship pada Neo4j juga dapat memiliki *property* yang menjelaskan informasi tambahan mengenai hubungan tersebut. Penentuan *relationship* dilakukan dengan memahami proses bisnis yang terjadi antar entitas. *Relationship* yang dirancang dengan baik dapat membantu sistem melakukan pencarian pola hubungan data yang lebih efektif.

3.3. Penentuan *Property*

Tahap berikutnya adalah menentukan *property* yang akan digunakan untuk menyimpan informasi tambahan. *Property* ini memiliki konsep yang mirip dengan atribut pada RDBMS. *Property* dapat dimiliki oleh *node* maupun *relationship*.

Penentuan ini dilakukan dengan mengidentifikasi informasi apa saja yang diperlukan dari setiap *node* maupun *relationship*. Pada *node*, *property* digunakan untuk menjelaskan karakteristik objek. Sedangkan pada *relationship*, *property* dapat digunakan untuk menjelaskan detail dari hubungan yang terjadi.

3.4.Perancangan *Graph* Model

Tahap terakhir adalah membuat rancangan *graph* model berdasarkan *node, relationship*, dan *property* yang telah ditentukan. *Graph* model digunakan sebagai representasi visual dari struktur *database* yang akan dibangun sehingga pengguna dapat memahami bagaimana data saling terhubung dalam sistem. Model yang telah dibuat kemudian dapat diimplementasikan ke dalam Neo4j menggunakan Cypher Query Language.

BAB IV
STUDI KASUS IMPLEMENTASI NEO4J

4.1. Deskripsi Studi Kasus

Pada eksplorasi ini, studi kasus yang diambil adalah Jaringan Riset Akademik, yaitu sebuah model sistem informasi yang menggambarkan hubungan antar entitas dalam ekosistem publikasi ilmiah. Sistem ini dirancang untuk merepresentasikan keterhubungan antara peneliti, institusi, publikasi ilmiah, topik penelitian, dan konferensi tempat penelitian dipublikasikan. Setiap elemen dalam jaringan memiliki hubungan yang saling berkaitan, seperti peneliti yang berafiliasi dengan institusi tertentu, *paper* yang membahas topik tertentu, serta hubungan sitasi antar publikasi ilmiah.

Kompleksitas hubungan dalam jaringan riset akademik membuat model *graph database* menjadi pendekatan yang sesuai untuk digunakan. Dengan pendekatan model *graph database*, sistem dapat mempermudah proses eksplorasi hubungan akademik, seperti menemukan peneliti dalam suatu institusi, melihat topik penelitian tertentu, atau menganalisis keterkaitan antar *paper* berdasarkan hubungan sitasi.

4.2. Model Data

Berdasarkan model data yang dirancang, sistem terdiri dari lima *node* utama, yaitu Researcher, Institution, Paper, Topic, dan Conference. *Node* tersebut merepresentasikan entitas utama dalam jaringan riset akademik, seperti peneliti, institusi asal, publikasi ilmiah, bidang penelitian, dan tempat publikasi *paper*. Hubungan antar *node* direpresentasikan menggunakan *relationship*, seperti AFFILIATED_WITH, AUTHORED, PUBLISHED_IN, COVERS, dan CITES yang menggambarkan keterkaitan antar entitas dalam sistem. Model *graph* ini memungkinkan proses pencarian dan analisis data berdasarkan hubungan antar entitas dilakukan secara langsung. Detail rancangan *graph diagram*, ERD, dan model relasional dapat dilihat pada lampiran melalui *repository* GitHub yang telah disediakan.

4.3. *Query* DML & *Read*

4.3.1. *Query* DML

1. Memperbarui status publikasi menjadi terverifikasi
2. Menambahkan relasi kolaborasi antar peneliti
3. Mengubah tahun konferensi tertentu
4. Menambahkan label status *paper* berdasarkan tahun publikasi

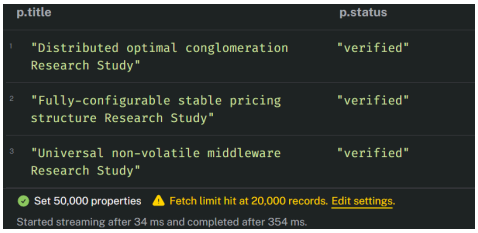
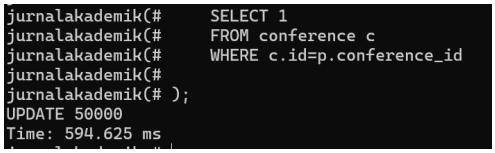
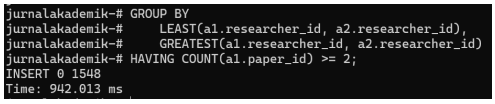
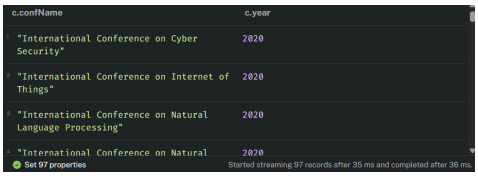
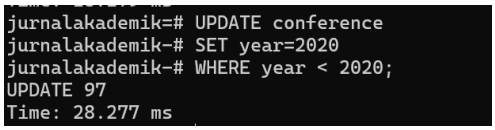
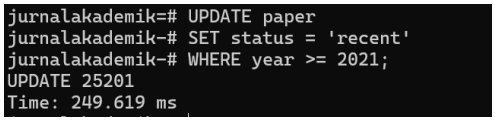
4.3.2. Query Read

1. Menampilkan riwayat publikasi seorang peneliti
2. Menampilkan peneliti dengan jumlah publikasi terbanyak
3. Mencari jaringan kolaborasi berdasarkan *paper* yang sama
4. Menampilkan daftar peneliti berdasarkan institusi
5. Menampilkan topik penelitian yang paling banyak dibahas
6. Menampilkan *paper* yang paling banyak disitasi

BAB V
ANALISIS PERBANDINGAN

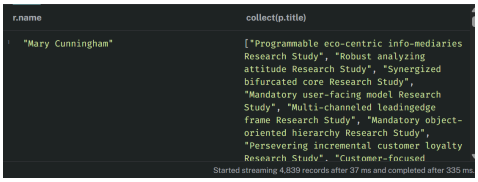
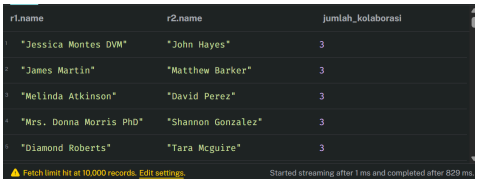
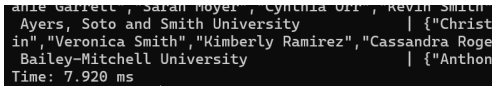
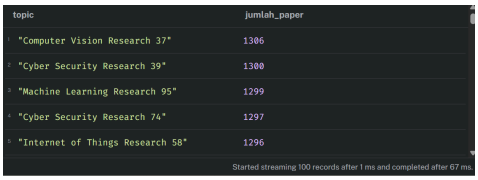
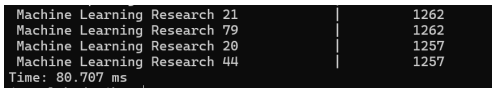
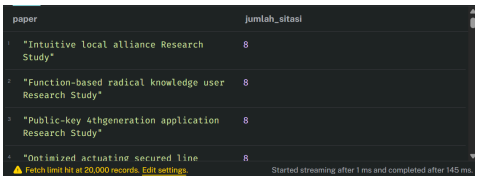
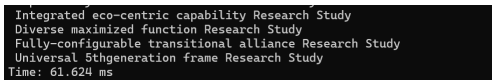
5.1. Neo4j vs *Relational Database* (PostgreSQL)

1. *Query* DML

Query	Neo4j	PostgreSQL
Query 1	 *Time*: 354 ms	 *Time*: 594,625 ms
Query 2	 *Time*: 566 ms	 *Time*: 942,013 ms
Query 3	 *Time*: 36 ms	 *Time*: 28,277 ms
Query 4	 *Time*: 191 ms	 *Time*: 249,619 ms

2. *Query Read*

Query	**Neo4j**	**PostgreSQL**

Query 1	 *Time*: 335 ms	 *Time*: 465,178 ms
Query 2	 *Time*: 137 ms	 *Time*: 73,278 ms
Query 3	 *Time*: 829 ms	 *Time*: 837,260 ms
Query 4	 *Time*: 8 ms	 *Time*: 7,920 ms
Query 5	 *Time*: 67 ms	 *Time*: 80,707 ms
Query 6	 *Time*: 145 ms	 *Time*: 61,624 ms

5.2. Analisis Perbandingan Neo4j dan PostgreSQL

Berdasarkan hasil implementasi dan pengujian menggunakan dataset jaringan riset akademik, terdapat beberapa perbedaan karakteristik antara Neo4j dan PostgreSQL.

5.2.1. Analisis Kinerja *Query*

Berdasarkan hasil pengujian, Neo4j dan PostgreSQL memiliki karakteristik performa yang berbeda tergantung jenis operasi yang dilakukan. Neo4j menunjukkan keunggulan pada *query* yang melibatkan eksplorasi hubungan antar entitas. Hal ini terlihat pada *query read* seperti pencarian riwayat publikasi, topik penelitian, dan hubungan sitasi yang memiliki waktu eksekusi lebih rendah dibandingkan PostgreSQL. Hal ini terjadi karena Neo4j menyimpan hubungan antar data secara langsung melalui *relationship* sehingga proses pencarian dilakukan melalui *graph traversal* tanpa membutuhkan proses JOIN antar tabel.

Namun, PostgreSQL masih memiliki performa yang baik pada operasi yang hanya melakukan manipulasi atribut sederhana. Pada operasi seperti perubahan nilai tahun konferensi, perbedaan waktu eksekusi dengan Neo4j relatif kecil karena *query* tidak membutuhkan pencarian hubungan yang kompleks. Hal ini menunjukkan bahwa Neo4j lebih unggul untuk data yang memiliki banyak hubungan, sedangkan PostgreSQL lebih cocok untuk data yang terstruktur.

5.2.2. Analisis Berdasarkan Pemodelan Data

Dari sisi pemodelan data, Neo4j lebih sesuai untuk studi kasus jaringan riset akademik karena hubungan antar entitas merupakan komponen utama sistem. Pada Neo4j, entitas seperti Researcher, Paper, Institution, Topic, dan Conference direpresentasikan sebagai *node*, sedangkan hubungan seperti AUTHORED, COVERS, dan CITES disimpan secara langsung sebagai *relationship*. Pendekatan ini membuat struktur data lebih mendekati kondisi nyata karena hubungan entitas dapat direpresentasikan tanpa tabel penghubung tambahan.

Pada PostgreSQL, struktur data harus dibuat dalam bentuk tabel dengan *foreign key* dan tabel *junction* untuk menangani relasi *many-to-many*. Pendekatan tersebut memiliki kelebihan dalam menjaga konsistensi data, tetapi membutuhkan proses JOIN ketika melakukan analisis hubungan. Oleh karena itu, PostgreSQL lebih cocok untuk sistem dengan struktur data yang stabil, sedangkan Neo4j lebih unggul untuk sistem yang membutuhkan analisis keterhubungan antar data.

5.2.3. Analisis Kompleksitas *Query* dan Fleksibilitas

Dari sisi *query*, Cypher pada Neo4j lebih sederhana dalam menggambarkan pola hubungan dibandingkan SQL pada PostgreSQL. *Query* untuk mencari kolaborasi antar peneliti dapat ditulis langsung berdasarkan pola hubungan antar *node*, sedangkan PostgreSQL membutuhkan beberapa operasi JOIN terhadap tabel relasi. Hal ini membuat Cypher lebih mudah digunakan untuk kasus yang memiliki banyak hubungan antar entitas.

Neo4j memiliki fleksibilitas lebih tinggi ketika terdapat penambahan jenis hubungan baru karena perubahan dapat dilakukan dengan menambahkan *node* atau *relationship* tanpa mengubah struktur tabel yang sudah ada. Namun, PostgreSQL memiliki keunggulan dalam pengelolaan data yang membutuhkan aturan integritas dan konsistensi yang ketat.

BAB VI
KESIMPULAN DAN SARAN

6.1. Kesimpulan

Berdasarkan hasil implementasi dan perbandingan antara Neo4j dengan PostgreSQL pada studi kasus jaringan riset akademik, dapat disimpulkan bahwa kedua DBMS memiliki kelebihan masing-masing. Neo4j lebih sesuai digunakan pada data yang memiliki banyak hubungan antar entitas karena hubungan disimpan secara langsung dalam bentuk *relationship*. Hal ini membuat proses pencarian hubungan seperti kolaborasi peneliti, topik penelitian, dan sitasi *paper* menjadi lebih sederhana.

Sementara itu, PostgreSQL lebih sesuai digunakan untuk data yang memiliki struktur tabel yang jelas dan membutuhkan pengelolaan data dengan aturan yang ketat. Dari hasil pengujian *query*, Neo4j memiliki performa yang lebih baik pada beberapa *query* yang melibatkan banyak hubungan, sedangkan PostgreSQL tetap dapat bekerja dengan baik pada operasi yang lebih sederhana. Dengan demikian, pemilihan DBMS perlu disesuaikan dengan kebutuhan sistem. Neo4j dapat menjadi pilihan yang sesuai untuk sistem yang berfokus pada analisis hubungan antar data dibandingkan *database* relasional.

6.2. Saran

Berdasarkan hasil eksplorasi yang telah dilakukan, beberapa pengembangan yang dapat dilakukan pada eksplorasi berikutnya adalah:

1. Melakukan pengujian menggunakan dataset yang lebih besar dengan jumlah *node* dan *relationship* yang lebih kompleks untuk mengetahui batas performa masing-masing DBMS.
2. Melakukan pengujian tambahan menggunakan fitur khusus *graph database* seperti algoritma Graph Data Science untuk analisis komunitas, *centrality*, dan pencarian pola hubungan.
3. Melakukan pengujian dengan skenario aplikasi nyata seperti sistem rekomendasi penelitian, pencarian kolaborator akademik, atau analisis hubungan publikasi.
4. Melakukan optimasi pada PostgreSQL menggunakan *indexing* dan optimasi *query* untuk memperoleh perbandingan yang lebih seimbang dengan Neo4j.

DAFTAR PUSTAKA

Neo4j, “What is Neo4j?,” *Neo4j Documentation*. Accessed: Oct. 11, 2026. [Online]. Available: https://neo4j.com/docs/getting-started/whats-neo4j/

Neo4j, “Open Source Project,” *Neo4j*. Accessed: Oct. 11, 2026. [Online]. Available: https://neo4j.com/open-source-project/

Neo4j, “Customer Stories,” *Neo4j*. Accessed: Oct. 11, 2026. [Online]. Available: https://neo4j.com/customer-stories/

Neo4j, “Data Modeling,” *Neo4j Documentation*. Accessed: Oct. 11, 2026. [Online]. Available: https://neo4j.com/docs/getting-started/data-modeling/

Neo4j, “Getting Started with Neo4j and JavaScript (Node.js),” *Neo4j Developer Resources*. Accessed: Oct. 11, 2026. [Online]. Available: https://neo4j.com/developer/languages/javascript/tutorials/getting-started-with-neo4j-and-javascript-node-js/

LAMPIRAN

Link Github: https://github.com/anellautari/project2-pdl-neo4j